

Essendo un algoritmo di ricerca dicotomica, la complessità è pari a $O(\log n)$.

```
int intruder(int[] A, int[] B, int n)
return intruderRec(A, B, 1, n + 1)
```

```
int intruderRec(int[] A, int[] B, int i, int j)
if i == j then
    return B[i]
else
    m = (i + j) / 2
    if A[m] == B[m] then
        return intruderRec(A, B, m + 1, j)
    else
        return intruderRec(A, B, i, m)
```

Esercizio B1

Questo problema non è altro che una versione ri-elaborata del problema dell'Insieme Indipendente di Intervalli. È sufficiente richiamare la funzione `independentSet()` presentata a lezione e restituire la dimensione dell'insieme così ottenuto. La complessità è $\Theta(n \log n)$, in quanto l'insieme di intervalli va ordinato per tempo di fine.

```
int longestChain(int[] X, int[] Y, int n)
SETS = independentSet(X, Y, n)
return S.size()
```

Esercizio B2

Sia $DP[i]$ il valore del più grande sottovettore di lunghezza pari che termina in posizione i . È possibile fare le seguenti osservazioni:

- Non esistono sottovettori di dimensione pari che terminano in posizione $i = 1$; quindi il valore è pari a 0.
- Nel caso di $i = 0$, non ho elementi, quindi il valore è di nuovo pari a zero.
- Nel caso generale, prendo il valore del più grande sottovettore che termina nella posizione $i - 2$ e sommo i valori $A[i - 1]$, $A[i]$. Se il risultato è positivo, vuol dire che posso estendere il sottovettore che termina in $i - 2$ con altri due elementi e il valore risultante è il più grande sottovettore che termina in posizione i ; se è negativo, il più grande sottovettore che termina in i è vuoto e il suo valore è 0.

Questo conduce alla seguente formulazione ricorsiva:

$$DP[i] = \begin{cases} 0 & i \leq 1 \\ \max\{DP[i - 2] + A[i - 1] + A[i], 0\} & i > 1 \end{cases}$$

A questo punto, il valore cercato è il valore massimo contenuto nella tabella DP – il più grande sottovettore che termina in qualsiasi indice del vettore.

Utilizziamo programmazione dinamica per tradurre questo formulazione ricorsiva in codice.

```

int maxSumEven(int[] A, int n)


---


    int[] DP = new int[0...n]
    DP[0] = DP[1] = 0
    for i = 2 to n do
        | DP[i] = max(DP[i - 2] + A[i - 1] + A[i], 0)
    return max(DP, n)


---



```

La complessità della soluzione è ovviamente lineare – $\theta(n)$.

Esercizio B3

Questo problema si risolve tramite backtrack - in effetti, non è altro che un problema di cammino più lungo su grafi, la cui versione decisionale è NP-completa.

Utilizziamo la matrice M sia come input, sia per memorizzare il fatto che una certa casella 1 è già stata visita, cambiando il valore associato da 1 a -1 . Si effettua una visita in profondità, aumentando di 1 la lunghezza ℓ del percorso trovato ad ogni passo successivo e confrontandolo con la variabile *maxSoFar*, passata per riferimento (è possibile risolvere il problema senza passare la variabile per riferimento, ma al prezzo di rendere l'algoritmo molto più difficile da leggere).

Un ulteriore trucco è quello di chiamare comunque la funzione ricorsivamente sulle quattro celle adiacenti, verificando, come primo step della chiamata, che la cella sia valida (i compreso fra 1 ed n , j compreso fra 1 e m) e che contenga un valore 1.

```

int longestPath(int[][] M, int n, int m)


---


    return longestPathRec(M, m, n, 1, 1)


---



```

```

int longestPathRec(int[][] M, int n, int m, int i, int j)


---


    if 1 ≤ i ≤ n and 1 ≤ j ≤ m and M[i][j] == 1 then
        | M[i][j] = -1                                     % Interpreted as "visited"
        | int max = 1 + max(
            |     longestPathRec(M, n, m, i + 1, j),
            |     longestPathRec(M, n, m, i, j + 1),
            |     longestPathRec(M, n, m, i - 1, j),
            |     longestPathRec(M, n, m, i, j - 1) )
        | M[i][j] = 1                                     % Not "visited" any more
        | return max
    else
        | return 0


---



```

La complessità dell'algoritmo è molto alta, $O(4^{nm})$.